Bouncer: Runtime Competence Auditing for Learned Systems Controllers

Kabir Grewal
ska@unc.edu

Abstract

Learned systems controllers can outperform fixed heuristics on their training distribution yet silently regress after workload shift. Input-distribution monitors cannot determine whether a controller's decisions still improve the system. We present BOUNCER, a runtime trust gate that measures *realized competence*: a secret sample of resource partitions runs the learned controller, another runs a vetted fallback, and a sequential detector gates the remaining partitions when the learned-minus-fallback reward falls below a threshold. This requires representative sampling, set-local reward, and policy contrast that survives assignment. In a new trained-controller case study, an offline logistic cache-insertion policy validates at 96.3% and initially beats LRU, but a label-semantic shift reverses its advantage while leaving its input histogram exactly unchanged. Across 12 seeds, BOUNCER detects all reversals in one audit window, retains 87.5% of the clean learned gain, and recovers 87.5% of the loss toward LRU. The study is controlled trace replay rather than full-system validation; separate ChampSim experiments expose where reward locality and stateful reassignment fail. The artifact is deterministic and released.

1 Why monitor competence?

Learned cache replacement, prefetching, memory scheduling, and branch prediction can beat hand-designed policies by adapting to workload structure [3, 7, 8, 12, 21]. Frameworks such as Park and SmartChoices make learned decisions available across systems domains [5, 14]. Their deployed advantage is nevertheless conditional: a model trained on one program or phase may become worse than the heuristic it displaced. This is a systems-specific instance of post-deployment model debt [19]. Aggregate counters show that performance changed, but not whether the learned policy caused the change. Feature-based out-of-distribution (OOD) detection is also indirect: a concept shift can preserve inputs while reversing which action is useful [4, 6, 18].

Suppose a learned controller C has a vetted fallback π_0. For audit window w, define the competence advantage

$$\Delta_w = \bar{r}_w^C - \bar{r}_w^{\pi_0}, \qquad r \in [0, r_{\max}], \tag{1}$$

where reward is the system outcome attributable to a decision, such as a cache hit. We want to deploy C while $\Delta_w \geq \tau$ and otherwise fall back. The counterfactual is unavailable: one physical decision cannot simultaneously run both policies.

This makes competence auditing an online causal-comparison problem, not a confidence-calibration problem. A useful mechanism must compare policies under concurrent traffic, spend only a small fraction of the resource on measurement, and state when its sample ceases to represent deployed decisions.

Contributions. We (i) define realized competence and an estimator with explicit validity conditions; (ii) decompose expected regret into detection, audit exposure, and switching; and (iii) evaluate one trained controller with independent training/validation traces, 12 seeded deployments, Student-t intervals, and checked numbers. ChampSim remains boundary evidence, not a performance win.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

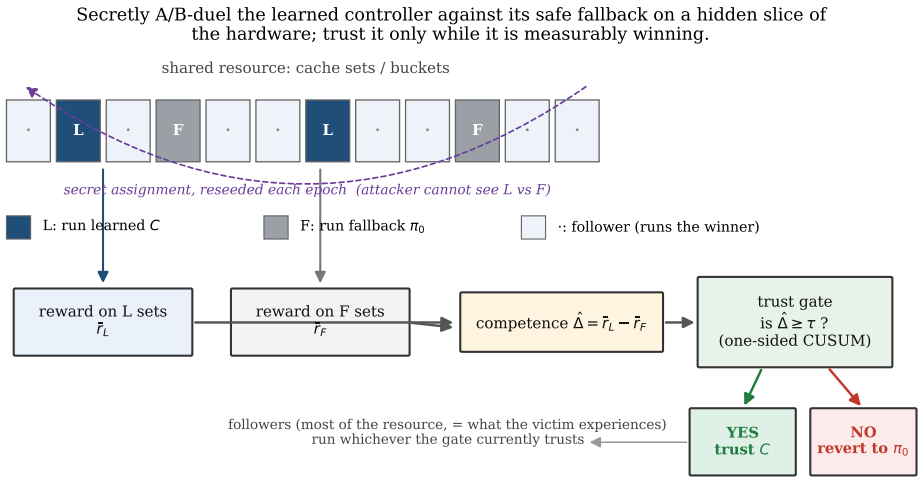

Figure 1: BOUNCER measures rather than predicts trust. Secret Leader-C partitions run the learned controller, Leader-F partitions run the fallback, and followers obey the gate. The mechanism is faithful only when the sampled reward represents deployed traffic and remains attributable to the sampled partition.

2 Secret online A/B auditing

Estimator and gate. BOUNCER repurposes hardware set dueling [17]. It secretly assigns n_C cache sets to C, n_F to π_0, and the rest to a follower pool (Figure 1). Unlike a global before/after comparison, both leaders observe the same concurrent workload. Their measured difference

$$\hat{\Delta}_w = \bar{r}_{C,w} - \bar{r}_{F,w} \tag{2}$$

feeds a lower CUSUM $S_w = \max\{0, S_{w-1} + K - \hat{\Delta}_w\}$, which gates followers when $S_w > H$ [16]. Hysteresis and measured probes support re-trust. Leaders remain policy-fixed so the gate continues to observe both arms.

Let Δ_w^{fol} be the decision-weighted advantage that the follower pool would experience. The estimator obeys $\mathbb{E}[\hat{\Delta}_w \mid \mathcal{H}_{w-1}] = \Delta_w^{\text{fol}} + b_w$, where b_w is the aggregate sampling/attribution/state bias. Sequential detection controls random fluctuation around this expectation; it cannot repair b_w. This distinction is why the conditions below are premises rather than implementation details.

When does the comparison mean what it says? Three conditions are load-bearing. **Representative sampling** requires the leader estimate to target decision-weighted follower traffic. **Set-locality** requires a decision and credited reward to share the sampled partition; replacement hits satisfy this, whereas a prefetch initiated in one set can benefit another. **Assignment-identifiability** requires the policy contrast to survive leader assignment. A stateless controller satisfies this immediately. A stateful controller instead needs persistent leaders, shadow-updated state, or a warm-up quarantine. Secret reshuffling that repeatedly cold-starts one arm can erase the very contrast being measured. Diagnostically, these conditions target $b_w = b_{\text{sample}} + b_{\text{local}} + b_{\text{state}} \approx 0$; the decomposition does not claim that its components are observable online.

Expected floor. The full paper proves the following compact accounting result. Over T windows, partition regret into (i) fully open harmful windows, (ii) the learned-policy exposure retained for auditing after gating, and (iii) false-alarm switching. If the expected number of fully open windows per harmful episode is at most D, each set is exposed after gating with marginal probability at most ϕ, the false-alarm probability per window is at most α, and each false-alarm episode costs at most c_{sw}, then

$$\mathbb{E}[R_T] \le N_{\text{ep}} D r_{\max} + \phi T_{\text{harm}} r_{\max} + \alpha T c_{\text{sw}}. \tag{3}$$

The result is expectation-level and conditional on the stated assumptions; it does not make a pointwise safety claim. In particular, D is an assumed or separately calibrated detection bound, not a latency

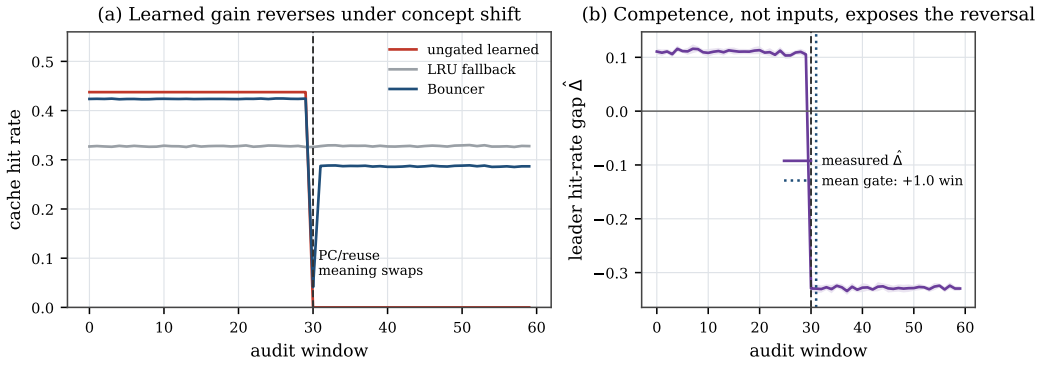

Figure 2: Mean and two-sided 95% Student-t intervals over 12 seeds. The trained insertion controller beats LRU before the semantic shift and collapses after it, although the PC histogram is unchanged. The measured leader gap reverses and gates followers one window later.

predicted by the inequality, and biased leaders invalidate its interpretation. Appendix B gives the proof skeleton and separates this accounting guarantee from the experiment.

3 One trained controller under concept shift

We add the smallest experiment missing from the original artifact: a controller whose deployment decisions come from fitted parameters. A logistic model sees one of four program-counter (PC) classes and predicts whether a line will be reused within 64 future references. Training and validation labels are computed from independent seeded traces containing recurring and one-shot lines. The controller inserts predicted-reusable lines and bypasses predicted-streaming lines; ordinary true LRU is the fallback. The model uses 20,000 training and 5,000 validation examples. Its fitted reuse probabilities for PCs 0–3 are $(0.937, 0.942, 0.046, 0.041)$; on the independent validation trace it achieves 96.28% accuracy versus a 50.82% majority baseline and 0.1603 log loss.

The deployment cache has 64 sets, eight ways, and 64 accesses per set-window. Across 60 windows, half the accesses target four recurring lines and half are one-shot streams. At window 30, the relationship between PC and future reuse reverses. Crucially, every PC still occurs exactly 25% of the time in every set-window, so a PC-histogram OOD detector has total-variation distance zero and never alarms. We use eight fixed secret leaders per arm and 12 independently seeded deployments. $K = 0.02$ and $H = 0.12$ are fixed across all runs. Seeds randomize leader placement and within-window access order inside one workload family; they are not 12 distinct application traces. Fixed leaders avoid a cache-state reset confound; this benign-shift study therefore does not test assignment secrecy against an adaptive attacker.

Figure 2 shows the causal reversal. Before the shift, hit rates are 0.4375 for the learned policy, 0.3276 for LRU, and 0.4237 for BOUNCER; hence BOUNCER retains 87.5% of the available learned gain. After the shift, they are 0, 0.3279, and 0.2869. All 12 runs gate one audit window after the shift and recover 87.5% of the learned-to-LRU loss; the two-sided 95% t intervals are [87.39,87.57]% retained and [87.46,87.51]% recovered. The remaining gap is exactly the fixed Leader-C exposure. These values are seed-level steady summaries that exclude five cold-start/transition windows per regime. They demonstrate the intended mechanism under a controlled concept shift, not generalization to production cache policies or an IPC benefit.

4 Broader evidence and boundaries

The trained case complements, rather than replaces, the artifact's broader tests. In the seeded competence harness, $\hat{\Delta}$ tracks known ground truth ($R^2 = 0.997$); the released operating point gates in two windows, limits the worst attacked steady window to 0.64% below the fallback, and detects 50/50 input-mimicry attacks while an input-OOD monitor detects 0/50. These are synthetic mechanism tests with explicit reward models.

The ChampSim integration supplies a different kind of evidence: it runs the auditor inside a cycle-level simulator on SPEC traces, but exposes two important boundaries rather than a clean learned-controller win. Prefetch reward is not set-local, and repeatedly reassigned replacement sets carry path-dependent cache state. Fixed replacement leaders recover a measurable gap, while reseeding can attenuate it. Thus the new trained trace replay establishes that fitted decisions can be gated; ChampSim establishes that the abstraction cannot be transferred blindly. Neither experiment measures RTL area, energy, timing, or real silicon. A 16-bit state-count proposal is 406 bytes, but it is not a synthesis result.

5 Related work and positioning

Learned systems controllers. RL memory scheduling and prefetching, oracle-imitation and neural cache replacement, and learned-systems platforms show that fitted decisions can beat heuristics [3, 5, 7, 8, 12, 14, 21]. These works ask how to construct or deploy a better policy. BOUNCER starts after a candidate and fallback exist and asks whether the candidate remains better on current traffic: it is an assurance wrapper, not another learned policy.

Shift and performance monitoring. OOD, two-sample, and concept-drift methods detect changes in inputs or representations [4, 6, 18]. Recent label-free methods more directly target harmful shift or model deterioration using learned error proxies or model disagreement [2, 15]. These are valuable when task labels or outcomes are delayed. A systems controller often produces an immediate local reward, so BOUNCER instead estimates a concurrent learned-minus-fallback reward. The methods are complementary: input monitoring can trigger more auditing, but an input alarm alone does not identify which policy caused a performance change.

Fallback assurance and online policy selection. Simplex, shielding, and constrained or safe policy-improvement methods retain a trusted controller or baseline when a state invariant or offline confidence condition is threatened [1, 11, 20, 22]. BOUNCER does not provide their constraint-safety guarantee; it supplies a conditional expected performance-regret bound from an online randomized comparison. Architecturally, DIP and its shared-cache extensions already dedicate sampled sets to competing cache policies [9, 10, 17]. We reuse that substrate but make the estimand, detector, and failure conditions explicit for a learned-versus-fallback competence decision. CUSUM supplies the sequential rule [13, 16]; it addresses delay/false-alarm tradeoffs, not biased or nonlocal reward.

6 Limitations and conclusion

The controlled controller is intentionally small, its PC/reuse reversal is constructed, and hit rate is not IPC. The 12 seeds vary randomization within a single workload family, so their narrow intervals quantify repeatability, not application diversity. The clean gap is far from the detector reference; zero false alarms here therefore does not calibrate a production false-alarm rate or show threshold robustness. Fixed leaders preserve state but provide less freshness than epoch reshuffling, and the trained experiment evaluates benign shift rather than a leader-aware adversary. More broadly, BOUNCER cannot faithfully audit nonlocal rewards, unrepresentative traffic, sub-resolution harm, or controllers whose relative behavior is destroyed by assignment. These are scope boundaries, not details hidden by the detector.

Within that scope, the result is encouraging: online randomized comparison can retain most of a trained controller's upside while recovering most of a known fallback when competence reverses, even when a simple input monitor has no signal. The released experiment and claim checker make that workshop-sized result directly reproducible, while the longer artifact retains the proofs, adaptive attacks, and full sensitivity studies. Next steps are multiple learned policies and application traces with separately calibrated thresholds.

References

[1] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*, pages 2669–2678. AAAI Press, 2018.

[2] Salim I. Amoukou, Tom Bewley, Saumitra Mishra, Freddy Lecue, Daniele Magazzeni, and Manuela Veloso. Sequential harmful shift detection without labels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2412.12910.

[3] Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1121–1137, 2021. doi: 10.1145/3466752.3480114.

[4] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):44:1–44:37, 2014. doi: 10.1145/2523813.

[5] Daniel Golovin, Gábor Bartók, Eric Chen, Emily Donahue, Tzu-Kuo Huang, Efi Kokiopoulou, Ruoyan Qin, Nikhil Sarda, Justin Sybrandt, and Vincent Tjeng. Smartchoices: Augmenting software with learned implementations. *arXiv preprint arXiv:2304.13033*, 2023.

[6] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[7] Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana. Self-optimizing memory controllers: A reinforcement learning approach. In *Proceedings of the 35th Annual International Symposium on Computer Architecture (ISCA)*, pages 39–50, 2008. doi: 10.1109/ISCA.2008.21.

[8] Akanksha Jain and Calvin Lin. Back to the future: Leveraging belady's algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pages 78–89, 2016. doi: 10.1109/ISCA.2016.17.

[9] Aamer Jaleel, William Hasenplaugh, Moinuddin K. Qureshi, Julien Sebot, Simon C. Steely, and Joel Emer. Adaptive insertion policies for managing shared caches. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 208–219. ACM, 2008. doi: 10.1145/1454115.1454145.

[10] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. High performance cache replacement using re-reference interval prediction (rrip). In *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*, pages 60–71. ACM, 2010. doi: 10.1145/1815961.1815971.

[11] Romain Laroche, Paul Trichelair, and Rémi Tachet des Combes. Safe policy improvement with baseline bootstrapping. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 3652–3661. PMLR, 2019.

[12] Evan Zheran Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. An imitation learning approach for cache replacement. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 6237–6247, 2020.

[13] G. Lorden. Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics*, 42(6):1897–1908, 1971. doi: 10.1214/aoms/1177693055.

[14] Hongzi Mao, Parimarjan Negi, Akshay Narayan, Hanrui Wang, Jiacheng Yang, Haonan Wang, Ryan Marcus, Ravichandra Addanki, Mehrdad Khani Shirkoohi, Songtao He, Vikram Nathan, Frank Cangialosi, Shaileshh Bojja Venkatakrishnan, Wei-Hung Weng, Song Han, Tim Kraska, and Mohammad Alizadeh. Park: An open platform for learning-augmented computer systems. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 2490–2502, 2019.

[15] Viet Nguyen, Changjian Shui, Vijay Giri, Siddharth Arya, Amol Verma, Fahad Razak, and Rahul G. Krishnan. Reliably detecting model failures in deployment without labels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. D3M; arXiv:2506.05047.

[16] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954. doi: 10.1093/biomet/41.1-2.100.

[17] Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Adaptive insertion policies for high performance caching. In *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, pages 381–391. ACM, 2007. doi: 10.1145/1250662.1250709.

[18] Stephan Rabanser, Stephan Günnemann, and Zachary C. Lipton. Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[19] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pages 2503–2511, 2015.

[20] Lui Sha. Using simplicity to control complexity. *IEEE Software*, 18(4):20–28, 2001. doi: 10.1109/MS.2001.936213.

[21] Zhan Shi, Xiangru Huang, Akanksha Jain, and Calvin Lin. Applying deep learning to the cache replacement problem. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 413–425, 2019. doi: 10.1145/3352460.3358319.

[22] Philip S. Thomas, Georgios Theocharous, and Mohammad Ghavamzadeh. High confidence policy improvement. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 2380–2388. PMLR, 2015.

A Trained-controller protocol

The released script `experiments/exp_trained_controller.py` owns the entire trained experiment. The model is a regularized binary logistic regressor with an intercept and four one-hot PC features. Full-batch gradient descent runs for 800 steps at learning rate 0.4 with 10^{-3} weight decay. Training traces contain equal numbers of recurring and one-shot references; 3% of examples cross the usual PC/reuse association. A label is one exactly when the same tag appears within the next 64 references. Training and validation traces use different seeds.

Deployment uses a set-associative trace replay with true LRU ordering. On a miss, the learned policy inserts the line if its fitted reuse probability is at least 0.5 and otherwise bypasses it. The fallback always inserts. Hot and stream references are balanced, as are all four PC classes. The PC classes associated with hot and stream references exchange at window 30. Hot tags rotate each window, preventing the learned controller from coasting indefinitely on state established before the shift.

Each evaluation seed draws one fixed secret partition: eight learned leaders, eight fallback leaders, and 48 followers. The one-sided lower CUSUM uses $K = 0.02$ and $H = 0.12$. Evidence from window w changes follower routing at window $w + 1$, so the reported one-window delay is causal rather than same-window routing. The experiment asserts detection in every seed, an unchanged PC histogram, a positive pre-shift learned advantage, a negative post-shift advantage, and minimum retained/recovered utility. The JSON and all manuscript numbers are checked by `scripts/check_ml4sys_numbers.py`. Seed-level intervals use the two-sided Student-t critical value 2.200985 for 11 degrees of freedom; no application-level population inference is intended.

Table 1: Evidence map. Each evaluation answers a different question.

Evidence	Strength	Deliberate boundary
Trained trace replay	Fitted controller; 12 seeded concept shifts	Controlled workload, hit rate rather than IPC
Synthetic harness	Known competence, adaptive attacks, exact claim checks	Parametric reward model
ChampSim/SPEC	Cycle-level execution and real cache state	Boundary study; committed logs, no clean learned-controller validation

B Expected-floor proof skeleton

Let $R_T = \sum_{w=1}^{T} (\bar{r}_w^{\pi_0} - \bar{r}_w^{\text{BOUNCER}})$. Split windows into three disjoint charges. During fully open harmful windows, bounded reward charges at most $r_{\max}$ per window; the assumed expected count is $N_{\text{ep}}D$. After gating, each harmful decision remains on C only through the audit exposure. Conditional on traffic and history, secret uniform sampling gives every set marginal exposure at most ϕ, so linearity of expectation charges at most $\phi T_{\text{harm}} r_{\max}$ without requiring independent traffic. Finally, the expected number of false alarms is at most αT, and each episode costs at most c_{sw}. Summing the three charges yields Equation 3; windows where C outperforms π_0 contribute nonpositive regret and may be discarded. The premise D includes both initial detection and any false re-trust. Only the exposure term has a separate high-probability refinement in the full artifact.

C Reproduction and claim ownership

Run `./run_ml4sys.sh`. It regenerates the training/evaluation JSON and figure, checks 15 headline values, compiles this PDF, and verifies that the main text ends by page four and references begin on page five. All random generators are explicitly seeded. The trained experiment requires NumPy and Matplotlib; the repository pins the complete environment. The full synthetic and ChampSim artifact remains available through `./run_all.sh`; the latter rebuilds figures from committed simulator logs and does not rerun every SPEC simulation.